

Lista de Exercícios — Implementando o Perceptron

Prof. Me. Gabriel Duarte

Disciplina: Inteligência Artificial

Objetivo

Implementar, **do zero** e em **NumPy puro**, o **Perceptron** de Rosenblatt e validá-lo nas **portas lógicas** OR, AND e XOR (as mesmas da nota de aula) e, como **Desafio**, num conjunto **real** (Iris). O foco é entender a **regra de correção de erro** $w \leftarrow w + \alpha(y - \hat{y})x$ & $b \leftarrow b + \alpha(y - \hat{y})$ — **manualmente, sem biblioteca de aprendizado**. Roda no **Google Colab** (CPU basta). Ao final, você deve enxergar com as próprias mãos: o Perceptron **converge** quando os dados são **linearmente separáveis** e **não converge** caso contrário (o famoso **XOR**).

1 Configuração e bases de dados

O Colab já traz `numpy` e `matplotlib` instalados. Para o Desafio, carregamos a base **Iris** apenas como *leitura de dados*:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
```

Regra importante na disciplina

“Sem biblioteca” aqui significa: **sem sklearn** (ou similar) para *ajustar o modelo*. O perceptron que você vai treinar precisa ser **escrito por você** (loop de épocas + regra de correção). **sklearn** pode aparecer **só** (a) para *carregar* a Iris, ou (b) **no final**, para conferir seu resultado contra o `sklearn.linear_model.Perceptron`. Usar **sklearn** para **treinar** o modelo zera a questão.

A porta lógica é um **dataset binário** de 4 amostras, com duas entradas $x_1, x_2 \in \{0, 1\}$ e um rótulo $y \in \{0, 1\}$:

x_1	x_2	OR (y)	AND (y)	XOR (y)
0	0	0	0	0
0	1	1	0	1
1	0	1	0	1
1	1	1	1	0

2 Exercício 1 — Porta OR (o caso que funciona)

Como funciona (recapitulando). O Perceptron calcula o potencial de ativação $z = w_1x_1 + w_2x_2 + b$ (a “balança” do porteiro) e decide com a função degrau: $\hat{y} = 1$ se $z \geq 0$, senão $\hat{y} = 0$. Se acertou, **não faz nada**; se errou, dá um “empurrão” nos pesos na direção que conserta o ponto:

$$w_j \leftarrow w_j + \alpha (y - \hat{y}) x_j, \quad b \leftarrow b + \alpha (y - \hat{y}).$$

1. Crie os vetores da OR: `X = np.array([[0,0],[0,1],[1,0],[1,1]], dtype=float)` e `y_or = np.array([0,1,1,1], dtype=float)`. Plote os 4 pontos ($x_1 \times x_2$), colorindo por classe (verde = classe 0, laranja = classe 1).
2. Implemente uma função `treina_perceptron(X, y, alpha=1.0, max_epocas=50)` que percorra as amostras **em ordem fixa**, aplique a regra acima *só* quando $y \neq \hat{y}$, e devolva `w`, `b`, e a lista `erros` (número de erros por época). Use $w = (0, 0)$ e $b = 0$ como ponto de partida.
3. Rode na OR. **Imprima** a lista `erros` e diga em que época o Perceptron **convergiu** (primeira época com 0 erros). Verifique que $\alpha = 1$ e $\hat{y} = 1$ quando $z \geq 0$ (não > 0).
4. Plote `erros` $\times$ época (gráfico de barras ou linha). Desenhe a **fronteira** final $w_1x_1 + w_2x_2 + b = 0$ sobre o gráfico dos pontos.

Antes de rodar

Rode o treino **manualmente na folha** para a ‘amostra (0,0) a (0,1) da época 0: começando em $w = (0,0), b = 0$, qual o z ? O Perceptron acerta ou erra? Que atualização ele faz? Depois compare com o que seu código imprimiu.

Dica de implementação

A atualização pode ser feita com `w = w + alpha * (y[i]-pred) * X[i]` e `b = b + alpha * (y[i]-pred)`. Como α multiplica o erro, só muda quando `pred != y[i]` (erro). Perceba que `y[i]-pred` vale +1 (era 0, devia ser 1) ou -1 (era 1, devia ser 0).

3 Exercício 2 — Porta AND

1. Crie `y_and = np.array([0,0,0,1], dtype=float)` e rode o mesmo `treina_perceptron`.
2. Registre em quantas épocas convergiu e a lista `erros`.
3. **Compare com a OR:** o AND precisou de **mais ou menos** épocas? Por quê, olhando para os pontos? Escreva 2–3 frases ligando a **distância dos pontos à fronteira inicial** com o número de atualizações.

4 Exercício 3 — A limitação do XOR (o caso que NÃO funciona)

Como funciona (recapitulando). Na aula vimos que o XOR tem as classes **entrelaçadas em cruz**: classe 0 em (0,0) e (1,1), classe 1 em (0,1) e (1,0). Por isso, **não existe** reta que os separe — o conjunto **não é linearmente separável**.

1. Crie `y_xor = np.array([0,1,1,0], dtype=float)` e rode o treino com `max_epocas = 50`.
2. O que acontece com a lista **erros** ao longo das épocas? O Perceptron **converge**? Registre a **acurácia** final (`np.mean(pred == y_xor)`).
3. (Oscilação) Rode de novo com `max_epocas = 1000`. O erro **some** em algum momento?
4. **Explique com suas palavras** (3–5 frases) por que *não* adianta treinar “por mais tempo” nem aumentar α : ligue o comportamento observado à definição de **linearmente separável** e ao **Teorema de Rosenblatt** (no fundo: o Perceptron **garante** convergência se, e somente se, houver separação linear).

5 Exercício 4 — (Desafio) Dataset real: Iris

Agora um problema que **funciona**, para contrastar com o XOR. Separe na Iris as classes **setosa** (rótulo 0) e **versicolor** (rótulo 1) usando **só duas características**: *comprimento da pétala* e *largura da pétala* (colunas 2 e 3).

```
from sklearn.datasets import load_iris
iris = load_iris()
Xb = iris.data[(iris.target==0) | (iris.target==1)][:, [2,3]]
yb = (iris.target[(iris.target==0) | (iris.target==1)] ==
1).astype(float)
```

1. Plote os pontos (x_2 = largura da pétala *vs* x_1 = comprimento) colorindo por classe. Os dois grupos são **linearmente separáveis**? Dá para “chutar” uma reta que os separe a olho?
2. Rode o `treina_perceptron` nessa base com $\alpha = 1$. Em quantas épocas convergiu? Qual a acurácia final? Anote w e b .
3. Desenhe a fronteira encontrada sobre o gráfico dos pontos.
4. **Reflexão final** (5 linhas): compare os **três** casos vistos — OR/AND (separáveis, convergem), XOR (não-separável, não converge) e Iris setosa/versicolor (separável, converge). O que a comparação te diz sobre *quando* o Perceptron é útil e sobre a ponte para redes multicamada (MLP)?

6 Entrega e critérios de avaliação

Você pode entregar em **duas formas** (escolha uma):

- um **notebook Jupyter** (‘.ipynb’) executado no Colab, comentado em PT-BR, ou
- um **script** ‘.py’ + as figuras geradas e os textos de resposta.

Em ambos, inclua as respostas escritas (itens que pedem texto) e as figuras.

Critério	Valor	Como é avaliado
Exercício 1 (OR)	2,5	Função <code>treina_perceptron</code> correta; convergiu; lista de erros e figura da fronteira
Exercício 2 (AND)	1,5	Convergiu; comparação OR vs AND bem explicada
Exercício 3 (XOR)	3,0	Mostrou que não converge; acurácia $\approx 0,5$; explicação correta (separabilidade)
Exercício 4 (Iris, Desafio)	2,0	Convergiu; figura; reflexão comparativa dos 3 casos
Organização, clareza e comentários	1,0	Código comentado em PT-BR, reproduzível no Colab

O que o gabarito espera (autochecagem)

OR: converge na **4ª época**, erros = [2,2,1,0], $w = (1,1)$, $b = -1$. AND: converge na **6ª época**, erros = [2,3,3,2,1,0], $w = (2,1)$, $b = -3$. XOR: **não converge**, acurácia $\approx 0,5$. Iris (setosa vs versicolor, pétala): converge na **3ª época**, $w = (0,5; 0,8)$, $b = -2$, acurácia 1,0. Se os seus números batem, **parabéns** — sua implementação está correta.